SYSTEM SPECIFICATION & TECHNICAL DOSSIER

Qyvaria DS Workbench & Engine

Comprehensive Architecture, Data Science Workflows, and Functional Core Implementation

1. Executive Overview

The Qyvaria DS Workbench & Open-Source Specification Engine is an integrated workspace designed for high-performance data science modeling, strict schema validation, and deterministic pipeline execution. This document compiles the core architectural specifications, runtime metrics, and foundational source code required to deploy and run the system effectively in production environments.

Data Science Workbench

Modular computational nodes featuring pluggable feature engineering transforms, automated missing-value handling, and cross-validation harnesses.

Open-Source Spec Engine

Declarative specification validator enforcing strict schema contracts, version-controlled metadata models, and automated compliance checking.

Operational Core Features

Designed for absolute predictability: zero external runtime dependencies in critical paths, pure functional transformations where applicable, and robust state serialization for seamless horizontal scaling across distributed clusters.

2. Core Pipeline Source Code

Python Engine Core

The functional core implementation handling modular data ingestion, deterministic feature scaling, and model evaluation hooks.

```
# qyvaria_core_engine.py
import json
import math
from typing import List, Dict, Any, Callable

class QyvariaPipeline:
    def __init__(self, config_path: str = None):
        self.config = self._load_config(config_path)
        self.transforms: List[Callable[[Dict[str, Any]], Dict[str, Any]]] = []
        self.metrics: Dict[str, float] = {}

    def _load_config(self, path: str) -> Dict[str, Any]:
        if not path:
            return {"version": "1.0.0", "strict_mode": True, "precision": 4}
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)

    def add_transform(self, func: Callable[[Dict[str, Any]], Dict[str, Any]]) -> None:
        self.transforms.append(func)

    def execute(self, dataset: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
        processed = []
        for row in dataset:
            current_row = row.copy()
            for transform in self.transforms:
                current_row = transform(current_row)
            processed.append(current_row)
        return processed

    def evaluate_model(self, y_true: List[float], y_pred: List[float]) -> float:
        if len(y_true) != len(y_pred) or not y_true:
            raise ValueError("Invalid target or prediction vectors.")
        mse = sum((t - p) ** 2 for t, p in zip(y_true, y_pred)) / len(y_true)
        self.metrics["mse"] = round(mse, self.config.get("precision", 4))
        return self.metrics["mse"]
```

Execution Guarantees

The code is fully self-contained, requiring zero third-party packages beyond standard Python libraries, guaranteeing deterministic parsing and evaluation across heterogeneous worker environments.

3. Specification & Schema Validation

JSON Spec Schema

Field Name	Type	Constraint	Default Behavior
`model_id`	String	UUID v4 format	Auto-generated
`learning_rate`	Float	0.0 < lr ≤ 1.0	0.0100
`validation_split`	Float	0.1 ≤ split ≤ 0.5	0.2000
`strict_contract`	Boolean	Exact match required	True

```
# specification_validator.py
def validate_specification(spec: Dict[str, Any]) -> bool:
    required_keys = ["model_id", "learning_rate", "validation_split"]
    for key in required_keys:
        if key not in spec:
            raise KeyError(f"Missing mandatory specification key: {key}")
    if not (0.0 < spec["learning_rate"] <= 1.0):
        raise ValueError("learning_rate out of bounds (0, 1].")
    return True
```

Deployment Note: The specification engine runs continuously as a background validator daemon, verifying schema integrity before any model training batch executes.